JavaScript is disabled on your browser.

Class MethodHandles.Lookup

java.lang.Object
    java.lang.invoke.MethodHandles.Lookup

Enclosing class:

MethodHandles

```
public static final class MethodHandles.Lookup
extends Object
```

A *lookup object* is a factory for creating method handles, when the creation requires access checking. Method handles do not perform access checks when they are called, but rather when they are created. Therefore, method handle access restrictions must be enforced when a method handle is created. The caller class against which those restrictions are enforced is known as the lookup class.

A lookup class which needs to create method handles will call `MethodHandles.lookup` to create a factory for itself. When the `Lookup` factory object is created, the identity of the lookup class is determined, and securely stored in the `Lookup` object. The lookup class (or its delegates) may then use factory methods on the `Lookup` object to create method handles for access-checked members. This includes all methods, constructors, and fields which are allowed to the lookup class, even private ones.

Lookup Factory Methods

The factory methods on a `Lookup` object correspond to all major use cases for methods, constructors, and fields. Each method handle created by a factory method is the functional equivalent of a particular *bytecode behavior*. (Bytecode behaviors are described in section 5.4.3.5 of the Java Virtual Machine Specification.) Here is a summary of the correspondence between these factory methods and the behavior the resulting method handles:

lookup expression	member	bytecode behavior

`lookup.findGetter(C.class,"f",FT.class)`	`FT f;`	`(T) this.f;`
`lookup.findStaticGetter(C.class,"f",FT.class)`	`static FT f;`	`(T) C.f;`
`lookup.findSetter(C.class,"f",FT.class)`	`FT f;`	`this.f = x;`
`lookup.findStaticSetter(C.class,"f",FT.class)`	`static FT f;`	`C.f = arg;`
`lookup.findVirtual(C.class,"m",MT)`	`T m(A*);`	`(T) this.m(arg*);`
`lookup.findStatic(C.class,"m",MT)`	`static T m(A*);`	`(T) C.m(arg*);`
`lookup.findSpecial(C.class,"m",MT,this.class)`	`T m(A*);`	`(T) super.m(arg*);`
`lookup.findConstructor(C.class,MT)`	`C(A*);`	`new C(arg*);`
`lookup.unreflectGetter(aField)`	`(static)? FT f;`	`(FT) aField.get(thisOrNull);`
`lookup.unreflectSetter(aField)`	`(static)? FT f;`	`aField.set(thisOrNull, arg);`
`lookup.unreflect(aMethod)`	`(static)? T m(A*);`	`(T) aMethod.invoke(thisOrNull, arg*);`
`lookup.unreflectConstructor(aConstructor)`	`C(A*);`	`(C) aConstructor.newInstance(arg*);`
`lookup.unreflect(aMethod)`	`(static)? T m(A*);`	`(T) aMethod.invoke(thisOrNull, arg*);`

Here, the type `C` is the class or interface being searched for a member, documented as a parameter named `refc` in the lookup methods. The method type `MT` is composed from the return type `T` and the sequence of argument types `A*`. The constructor also has a sequence of argument types `A*` and is deemed to return the newly-created object of type `C`. Both `MT` and the field type `FT` are documented as a parameter named `type`. The formal parameter `this` stands for the self-reference of type `C`; if it is present, it is always the leading argument to the method handle invocation. (In the case of some `protected` members, `this` may be restricted in type to the lookup class; see below.) The name `arg` stands for all the other method handle arguments. In the code examples for the Core Reflection API, the name `thisOrNull` stands for a null reference if the accessed method or field is static, and `this` otherwise. The names `aMethod`, `aField`, and `aConstructor` stand for reflective objects corresponding to the given members.

In cases where the given member is of variable arity (i.e., a method or constructor) the returned method handle will also be of variable arity. In all other cases, the returned method handle will be of fixed arity.

Discussion: The equivalence between looked-up method handles and underlying class members and bytecode behaviors can break down in a few ways:

- If `C` is not symbolically accessible from the lookup class's loader, the lookup can still succeed, even when there is no equivalent Java expression or bytecoded constant.
- Likewise, if `T` or `MT` is not symbolically accessible from the lookup class's loader, the lookup can still succeed. For example, lookups for `MethodHandle.invokeExact` and `MethodHandle.invoke` will always succeed, regardless of requested type.

- If there is a security manager installed, it can forbid the lookup on various grounds (see below). By contrast, the `ldc` instruction on a `CONSTANT_MethodHandle` constant is not subject to security manager checks.
- If the looked-up method has a very large arity, the method handle creation may fail, due to the method handle type having too many parameters.

Access checking

Access checks are applied in the factory methods of `Lookup`, when a method handle is created. This is a key difference from the Core Reflection API, since `java.lang.reflect.Method.invoke` performs access checking against every caller, on every call.

All access checks start from a `Lookup` object, which compares its recorded lookup class against all requests to create method handles. A single `Lookup` object can be used to create any number of access-checked method handles, all checked against a single lookup class.

A `Lookup` object can be shared with other trusted code, such as a metaobject protocol. A shared `Lookup` object delegates the capability to create method handles on private members of the lookup class. Even if privileged code uses the `Lookup` object, the access checking is confined to the privileges of the original lookup class.

A lookup can fail, because the containing class is not accessible to the lookup class, or because the desired class member is missing, or because the desired class member is not accessible to the lookup class, or because the lookup object is not trusted enough to access the member. In any of these cases, a `ReflectiveOperationException` will be thrown from the attempted lookup. The exact class will be one of the following:

- NoSuchMethodException — if a method is requested but does not exist
- NoSuchFieldException — if a field is requested but does not exist
- IllegalAccessException — if the member exists but an access check fails

In general, the conditions under which a method handle may be looked up for a method `M` are no more restrictive than the conditions under which the lookup class could have compiled, verified, and resolved a call to `M`. Where the JVM would raise exceptions like `NoSuchMethodError`, a method handle lookup will generally raise a corresponding checked exception, such as `NoSuchMethodException`. And the effect of invoking the method handle resulting from the lookup is exactly equivalent to executing the compiled, verified, and resolved call to `M`. The same point is true of fields and constructors.

Discussion: Access checks only apply to named and reflected methods, constructors, and fields. Other method handle creation methods, such as `MethodHandle.asType`, do not require any access checks, and are used independently of any `Lookup` object.

If the desired member is `protected`, the usual JVM rules apply, including the requirement that the lookup class must be either be in the same package as the desired member, or must inherit that member. (See the Java Virtual Machine Specification, sections 4.9.2, 5.4.3.5, and 6.4.) In addition, if the desired member is

a non-static field or method in a different package, the resulting method handle may only be applied to objects of the lookup class or one of its subclasses. This requirement is enforced by narrowing the type of the leading `this` parameter from `C` (which will necessarily be a superclass of the lookup class) to the lookup class itself.

The JVM imposes a similar requirement on `invokespecial` instruction, that the receiver argument must match both the resolved method *and* the current class. Again, this requirement is enforced by narrowing the type of the leading parameter to the resulting method handle. (See the Java Virtual Machine Specification, section 4.10.1.9.)

The JVM represents constructors and static initializer blocks as internal methods with special names (`"<init>"` and `"<clinit>"`). The internal syntax of invocation instructions allows them to refer to such internal methods as if they were normal methods, but the JVM bytecode verifier rejects them. A lookup of such an internal method will produce a `NoSuchMethodException`.

In some cases, access between nested classes is obtained by the Java compiler by creating an wrapper method to access a private method of another class in the same top-level declaration. For example, a nested class `C.D` can access private members within other related classes such as `C`, `C.D.E`, or `C.B`, but the Java compiler may need to generate wrapper methods in those related classes. In such cases, a `Lookup` object on `C.E` would be unable to those private members. A workaround for this limitation is the `Lookup.in` method, which can transform a lookup on `C.E` into one on any of those other classes, without special elevation of privilege.

The accesses permitted to a given lookup object may be limited, according to its set of `lookupModes`, to a subset of members normally accessible to the lookup class. For example, the `publicLookup` method produces a lookup object which is only allowed to access public members in public classes. The caller sensitive method `lookup` produces a lookup object with full capabilities relative to its caller class, to emulate all supported bytecode behaviors. Also, the `Lookup.in` method may produce a lookup object with fewer access modes than the original lookup object.

Discussion of private access: We say that a lookup has *private access* if its lookup modes include the possibility of accessing `private` members. As documented in the relevant methods elsewhere, only lookups with private access possess the following capabilities:

- access private fields, methods, and constructors of the lookup class
- create method handles which invoke caller sensitive methods, such as `Class.forName`
- create method handles which `emulate invokespecial` instructions
- avoid package access checks for classes accessible to the lookup class
- create `delegated lookup objects` which have private access to other classes within the same package member

Each of these permissions is a consequence of the fact that a lookup object with private access can be securely traced back to an originating class, whose bytecode behaviors and Java language access permissions can be reliably determined and emulated by method handles.

Security manager interactions

Although bytecode instructions can only refer to classes in a related class loader, this API can search for methods in any class, as long as a reference to its `Class` object is available. Such cross-loader references are also possible with the Core Reflection API, and are impossible to bytecode instructions such as `invokestatic` or `getfield`. There is a security manager API to allow applications to check such cross-loader references. These checks apply to both the `MethodHandles.Lookup` API and the Core Reflection API (as found on `Class`).

If a security manager is present, member lookups are subject to additional checks. From one to three calls are made to the security manager. Any of these calls can refuse access by throwing a `SecurityException`. Define `smgr` as the security manager, `lookc` as the lookup class of the current lookup object, `refc` as the containing class in which the member is being sought, and `defc` as the class in which the member is actually defined. The value `lookc` is defined as *not present* if the current lookup object does not have private access. The calls are made according to the following rules:

- **Step 1:** If `lookc` is not present, or if its class loader is not the same as or an ancestor of the class loader of `refc`, then `smgr.checkPackageAccess(refcPkg)` is called, where `refcPkg` is the package of `refc`.
- **Step 2:** If the retrieved member is not public and `lookc` is not present, then `smgr.checkPermission` with `RuntimePermission("accessDeclaredMembers")` is called.
- **Step 3:** If the retrieved member is not public, and if `lookc` is not present, and if `defc` and `refc` are different, then `smgr.checkPackageAccess(defcPkg)` is called, where `defcPkg` is the package of `defc`.

Security checks are performed after other access checks have passed. Therefore, the above rules presuppose a member that is public, or else that is being accessed from a lookup class that has rights to access the member.

Caller sensitive methods

A small number of Java methods have a special property called caller sensitivity. A *caller-sensitive* method can behave differently depending on the identity of its immediate caller.

If a method handle for a caller-sensitive method is requested, the general rules for bytecode behaviors apply, but they take account of the lookup class in a special way. The resulting method handle behaves as if it were called from an instruction contained in the lookup class, so that the caller-sensitive method detects the lookup class. (By contrast, the invoker of the method handle is disregarded.) Thus, in the case of caller-sensitive methods, different lookup classes may give rise to differently behaving method handles.

In cases where the lookup object is `publicLookup()`, or some other lookup object without private access, the lookup class is disregarded. In such cases, no caller-

sensitive method handle can be created, access is forbidden, and the lookup fails with an `IllegalAccessException`.

Discussion: For example, the caller-sensitive method `Class.forName(x)` can return varying classes or throw varying exceptions, depending on the class loader of the class that calls it. A public lookup of `Class.forName` will fail, because there is no reasonable way to determine its bytecode behavior.

If an application caches method handles for broad sharing, it should use `publicLookup()` to create them. If there is a lookup of `Class.forName`, it will fail, and the application must take appropriate action in that case. It may be that a later lookup, perhaps during the invocation of a bootstrap method, can incorporate the specific identity of the caller, making the method accessible.

The function `MethodHandles.lookup` is caller sensitive so that there can be a secure foundation for lookups. Nearly all other methods in the JSR 292 API rely on lookup objects to check access requests.

Field Summary

Fields

Modifier and Type	Field	Description
`static int`	**`PACKAGE`**	A single-bit mask representing `package` access (default access), which may contribute to the result of **`lookupModes`**.
`static int`	**`PRIVATE`**	A single-bit mask representing `private` access, which may contribute to the result of **`lookupModes`**.
`static int`	**`PROTECTED`**	A single-bit mask representing `protected` access, which may contribute to the result of **`lookupModes`**.
`static int`	**`PUBLIC`**	A single-bit mask representing `public` access, which may contribute to the result of **`lookupModes`**.

Method Summary

Concrete Methods

Modifier and Type	Method	Description

MethodHandle	**bind**(**Object** receiver, **String** name, **MethodType** type)	Produces an early-bound method handle for a non-static method.
MethodHandle	**findConstructor**(**Class**<?> refc, **MethodType** type)	Produces a method handle which creates an object and initializes it, using the constructor of the specified type.
MethodHandle	**findGetter**(**Class**<?> refc, **String** name, **Class**<?> type)	Produces a method handle giving read access to a non-static field.
MethodHandle	**findSetter**(**Class**<?> refc, **String** name, **Class**<?> type)	Produces a method handle giving write access to a non-static field.
MethodHandle	**findSpecial**(**Class**<?> refc, **String** name, **MethodType** type, **Class**<?> specialCaller)	Produces an early-bound method handle for a virtual method.
MethodHandle	**findStatic**(**Class**<?> refc, **String** name, **MethodType** type)	Produces a method handle for

		a static method.
MethodHandle	**findStaticGetter**(**Class**<?> refc, **String** name, **Class**<?> type)	Produces a method handle giving read access to a static field.
MethodHandle	**findStaticSetter**(**Class**<?> refc, **String** name, **Class**<?> type)	Produces a method handle giving write access to a static field.
MethodHandle	**findVirtual**(**Class**<?> refc, **String** name, **MethodType** type)	Produces a method handle for a virtual method.
MethodHandles.Lookup	**in**(**Class**<?> requestedLookupClass)	Creates a lookup on the specified new lookup class.
Class<?>	**lookupClass**()	Tells which class is performing the lookup.
int	**lookupModes**()	Tells which access-protection classes of members this lookup object can produce.
MethodHandleInfo	**revealDirect**(**MethodHandle** target)	Cracks a **direct method handle** created by

		this lookup object or a similar one.
String	**toString**()	Displays the name of the class from which lookups are to be made.
MethodHandle	**unreflect**(**Method** m)	Makes a **direct method handle** to *m*, if the lookup class has permission.
MethodHandle	**unreflectConstructor**(**Constructor**<?> c)	Produces a method handle for a reflected constructor.
MethodHandle	**unreflectGetter**(**Field** f)	Produces a method handle giving read access to a reflected field.
MethodHandle	**unreflectSetter**(**Field** f)	Produces a method handle giving write access to a reflected field.
MethodHandle	**unreflectSpecial**(**Method** m, **Class**<?> specialCaller)	Produces a method handle for a reflected method.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Field Detail

PUBLIC

```
public static final int PUBLIC
```

A single-bit mask representing `public` access, which may contribute to the result of `lookupModes`. The value, `0x01`, happens to be the same as the value of the `public` modifier bit.

See Also:

Constant Field Values

PRIVATE

```
public static final int PRIVATE
```

A single-bit mask representing `private` access, which may contribute to the result of `lookupModes`. The value, `0x02`, happens to be the same as the value of the `private` modifier bit.

See Also:

Constant Field Values

PROTECTED

```
public static final int PROTECTED
```

A single-bit mask representing `protected` access, which may contribute to the result of `lookupModes`. The value, `0x04`, happens to be the same as the value of the `protected` modifier bit.

See Also:

Constant Field Values

PACKAGE

```
public static final int PACKAGE
```

A single-bit mask representing `package` access (default access), which may contribute to the result of `lookupModes`. The value is `0x08`, which does not correspond meaningfully to any particular modifier bit.

See Also:

`Constant Field Values`

Method Detail

lookupClass

```
public Class<?> lookupClass()
```

Tells which class is performing the lookup. It is this class against which checks are performed for visibility and access permissions.

The class implies a maximum level of access permission, but the permissions may be additionally limited by the bitmask `lookupModes`, which controls whether non-public members can be accessed.

Returns:

`the lookup class, on behalf of which this lookup object finds members`

lookupModes

```
public int lookupModes()
```

Tells which access-protection classes of members this lookup object can produce. The result is a bit-mask of the bits PUBLIC (0x01), PRIVATE (0x02), PROTECTED (0x04), and PACKAGE (0x08).

A freshly-created lookup object on the caller's class has all possible bits set, since the caller class can access all its own members. A lookup object on a new lookup class created from a previous lookup object may have some mode bits set to zero. The purpose of this is to restrict access via the new lookup object, so that it can access only names which can be reached by the original lookup object, and also by the new lookup class.

Returns:

`the lookup modes, which limit the kinds of access performed by this lookup object`

in

```
public MethodHandles.Lookup in(Class<?> requestedLookupClass)
```

Creates a lookup on the specified new lookup class. The resulting object will report the specified class as its own `lookupClass`.

However, the resulting `Lookup` object is guaranteed to have no more access capabilities than the original. In particular, access capabilities can be lost as follows:

- If the new lookup class differs from the old one, protected members will not be accessible by virtue of inheritance. (Protected members may continue to be accessible because of package sharing.)
- If the new lookup class is in a different package than the old one, protected and default (package) members will not be accessible.
- If the new lookup class is not within the same package member as the old one, private members will not be accessible.
- If the new lookup class is not accessible to the old lookup class, then no members, not even public members, will be accessible. (In all other cases, public members will continue to be accessible.)

Parameters:

`requestedLookupClass - the desired lookup class for the new lookup object`

Returns:

`a lookup object which reports the desired lookup class`

Throws:

`NullPointerException - if the argument is null`

toString

```
public String toString()
```

Displays the name of the class from which lookups are to be made. (The name is the one reported by `Class.getName`.) If there are restrictions on the access permitted to this lookup, this is indicated by adding a suffix to the class name, consisting of a slash and a keyword. The keyword represents the strongest allowed access, and is chosen as follows:

- If no access is allowed, the suffix is "/noaccess".
- If only public access is allowed, the suffix is "/public".
- If only public and package access are allowed, the suffix is "/package".
- If only public, package, and private access are allowed, the suffix is "/private".

If none of the above cases apply, it is the case that full access (public, package, private, and protected) is allowed. In this case, no suffix is added. This is true only of an object obtained originally from

MethodHandles.lookup. Objects created by Lookup.in always have restricted access, and will display a suffix.

(It may seem strange that protected access should be stronger than private access. Viewed independently from package access, protected access is the first to be lost, because it requires a direct subclass relationship between caller and callee.)

Overrides:

toString in class Object

Returns:

a string representation of the object.

See Also:

in(java.lang.Class<?>)

findStatic

```
public MethodHandle findStatic(Class<?> refc,
                               String name,
                               MethodType type)
                        throws NoSuchMethodException,
                               IllegalAccessException
```

Produces a method handle for a static method. The type of the method handle will be that of the method. (Since static methods do not take receivers, there is no additional receiver argument inserted into the method handle type, as there would be with findVirtual or findSpecial.) The method and all its argument types must be accessible to the lookup object.

The returned method handle will have variable arity if and only if the method's variable arity modifier bit (0x0080) is set.

If the returned method handle is invoked, the method's class will be initialized, if it has not already been initialized.

Example:

```
import static java.lang.invoke.MethodHandles.*;
import static java.lang.invoke.MethodType.*;
...
MethodHandle MH_asList = publicLookup().findStatic(Arrays.class,
  "asList", methodType(List.class, Object[].class));
assertEquals("[x, y]", MH_asList.invoke("x", "y").toString());
```

Parameters:

refc - the class from which the method is accessed

name - the name of the method

type - the type of the method

Returns:

the desired method handle

Throws:

NoSuchMethodException - if the method does not exist

IllegalAccessException - if access checking fails, or if the method is not static, or if the method's variable arity modifier bit is set and asVarargsCollector fails

SecurityException - if a security manager is present and it refuses access

NullPointerException - if any argument is null

findVirtual

```
public MethodHandle findVirtual(Class<?> refc,
                                String name,
                                MethodType type)
                         throws NoSuchMethodException,
                                IllegalAccessException
```

Produces a method handle for a virtual method. The type of the method handle will be that of the method, with the receiver type (usually `refc`) prepended. The method and all its argument types must be accessible to the lookup object.

When called, the handle will treat the first argument as a receiver and dispatch on the receiver's type to determine which method implementation to enter. (The dispatching action is identical with that performed by an `invokevirtual` or `invokeinterface` instruction.)

The first argument will be of type `refc` if the lookup class has full privileges to access the member. Otherwise the member must be `protected` and the first argument will be restricted in type to the lookup class.

The returned method handle will have variable arity if and only if the method's variable arity modifier bit (`0x0080`) is set.

Because of the general equivalence between `invokevirtual` instructions and method handles produced by `findVirtual`, if the class is `MethodHandle` and the name string is `invokeExact` or `invoke`, the resulting method handle is equivalent to one produced by `MethodHandles.exactInvoker` or `MethodHandles.invoker` with the same `type` argument. **Example:**

```
    import static java.lang.invoke.MethodHandles.*;
```

```
import static java.lang.invoke.MethodType.*;
...
MethodHandle MH_concat = publicLookup().findVirtual(String.class,
  "concat", methodType(String.class, String.class));
MethodHandle MH_hashCode = publicLookup().findVirtual(Object.class,
  "hashCode", methodType(int.class));
MethodHandle MH_hashCode_String = publicLookup().findVirtual(String.class,
  "hashCode", methodType(int.class));
assertEquals("xy", (String) MH_concat.invokeExact("x", "y"));
assertEquals("xy".hashCode(), (int) MH_hashCode.invokeExact((Object)"xy"));
assertEquals("xy".hashCode(), (int) MH_hashCode_String.invokeExact("xy"));
// interface method:
MethodHandle MH_subSequence = publicLookup().findVirtual(CharSequence.class,
  "subSequence", methodType(CharSequence.class, int.class, int.class));
assertEquals("def", MH_subSequence.invoke("abcdefghi", 3, 6).toString());
// constructor "internal method" must be accessed differently:
MethodType MT_newString = methodType(void.class); //()V for new String()
try { assertEquals("impossible", lookup()
        .findVirtual(String.class, "<init>", MT_newString));
 } catch (NoSuchMethodException ex) { } // OK
MethodHandle MH_newString = publicLookup()
  .findConstructor(String.class, MT_newString);
assertEquals("", (String) MH_newString.invokeExact());
```

Parameters:

refc - the class or interface from which the method is accessed

name - the name of the method

type - the type of the method, with the receiver argument omitted

Returns:

the desired method handle

Throws:

NoSuchMethodException - if the method does not exist

IllegalAccessException - if access checking fails, or if the method is static or if the method's variable arity modifier bit is set and asVarargsCollector fails

SecurityException - if a security manager is present and it refuses access

NullPointerException - if any argument is null

findConstructor

```
public MethodHandle findConstructor(Class<?> refc,
                                    MethodType type)
```

```
                                throws NoSuchMethodException,
                                       IllegalAccessException
```

Produces a method handle which creates an object and initializes it, using the constructor of the specified type. The parameter types of the method handle will be those of the constructor, while the return type will be a reference to the constructor's class. The constructor and all its argument types must be accessible to the lookup object.

The requested type must have a return type of `void`. (This is consistent with the JVM's treatment of constructor type descriptors.)

The returned method handle will have variable arity if and only if the constructor's variable arity modifier bit (`0x0080`) is set.

If the returned method handle is invoked, the constructor's class will be initialized, if it has not already been initialized.

Example:

```
import static java.lang.invoke.MethodHandles.*;
import static java.lang.invoke.MethodType.*;
...
MethodHandle MH_newArrayList = publicLookup().findConstructor(
  ArrayList.class, methodType(void.class, Collection.class));
Collection orig = Arrays.asList("x", "y");
Collection copy = (ArrayList) MH_newArrayList.invokeExact(orig);
assert(orig != copy);
assertEquals(orig, copy);
// a variable-arity constructor:
MethodHandle MH_newProcessBuilder = publicLookup().findConstructor(
  ProcessBuilder.class, methodType(void.class, String[].class));
ProcessBuilder pb = (ProcessBuilder)
  MH_newProcessBuilder.invoke("x", "y", "z");
assertEquals("[x, y, z]", pb.command().toString());
```

Parameters:

`refc - the class or interface from which the method is accessed`

`type - the type of the method, with the receiver argument omitted, and a void return type`

Returns:

`the desired method handle`

Throws:

`NoSuchMethodException - if the constructor does not exist`

`IllegalAccessException - if access checking fails or if the method's variable arity modifier bit is set and asVarargsCollector fails`

SecurityException - if a security manager is present and it refuses access

NullPointerException - if any argument is null

findSpecial

```
public MethodHandle findSpecial(Class<?> refc,
                                String name,
                                MethodType type,
                                Class<?> specialCaller)
                         throws NoSuchMethodException,
                                IllegalAccessException
```

Produces an early-bound method handle for a virtual method. It will bypass checks for overriding methods on the receiver, as if called from an `invokespecial` instruction from within the explicitly specified `specialCaller`. The type of the method handle will be that of the method, with a suitably restricted receiver type prepended. (The receiver type will be `specialCaller` or a subtype.) The method and all its argument types must be accessible to the lookup object.

Before method resolution, if the explicitly specified caller class is not identical with the lookup class, or if this lookup object does not have private access privileges, the access fails.

The returned method handle will have variable arity if and only if the method's variable arity modifier bit (`0x0080`) is set.

(Note: JVM internal methods named "`<init>`" are not visible to this API, even though the `invokespecial` instruction can refer to them in special circumstances. Use `findConstructor` to access instance initialization methods in a safe manner.)

Example:

```
import static java.lang.invoke.MethodHandles.*;
import static java.lang.invoke.MethodType.*;
...
static class Listie extends ArrayList {
  public String toString() { return "[wee Listie]"; }
  static Lookup lookup() { return MethodHandles.lookup(); }
}
...
// no access to constructor via invokeSpecial:
MethodHandle MH_newListie = Listie.lookup()
  .findConstructor(Listie.class, methodType(void.class));
Listie l = (Listie) MH_newListie.invokeExact();
try { assertEquals("impossible", Listie.lookup().findSpecial(
        Listie.class, "<init>", methodType(void.class), Listie.class));
```

```
     } catch (NoSuchMethodException ex) { } // OK
    // access to super and self methods via invokeSpecial:
    MethodHandle MH_super = Listie.lookup().findSpecial(
      ArrayList.class, "toString" , methodType(String.class), Listie.class);
    MethodHandle MH_this = Listie.lookup().findSpecial(
      Listie.class, "toString" , methodType(String.class), Listie.class);
    MethodHandle MH_duper = Listie.lookup().findSpecial(
      Object.class, "toString" , methodType(String.class), Listie.class);
    assertEquals("[]", (String) MH_super.invokeExact(l));
    assertEquals(""+l, (String) MH_this.invokeExact(l));
    assertEquals("[]", (String) MH_duper.invokeExact(l)); // ArrayList method
    try { assertEquals("inaccessible", Listie.lookup().findSpecial(
            String.class, "toString", methodType(String.class), Listie.class));
     } catch (IllegalAccessException ex) { } // OK
    Listie subl = new Listie() { public String toString() { return "[subclass]";
    assertEquals(""+l, (String) MH_this.invokeExact(subl)); // Listie method
```

Parameters:

refc - the class or interface from which the method is accessed

name - the name of the method (which must not be "<init>")

type - the type of the method, with the receiver argument omitted

specialCaller - the proposed calling class to perform the invokespecial

Returns:

the desired method handle

Throws:

NoSuchMethodException - if the method does not exist

IllegalAccessException - if access checking fails or if the method's variable arity modifier bit is set and asVarargsCollector fails

SecurityException - if a security manager is present and it refuses access

NullPointerException - if any argument is null

findGetter

```
public MethodHandle findGetter(Class<?> refc,
                               String name,
                               Class<?> type)
                        throws NoSuchFieldException,
                               IllegalAccessException
```

Produces a method handle giving read access to a non-static field. The type of the method handle will have a return type of the field's value type. The method handle's single argument will be the instance containing the field. Access checking is performed immediately on behalf of the lookup class.

Parameters:

`refc - the class or interface from which the method is accessed`

`name - the field's name`

`type - the field's type`

Returns:

`a method handle which can load values from the field`

Throws:

`NoSuchFieldException - if the field does not exist`

`IllegalAccessException - if access checking fails, or if the field is static`

`SecurityException - if a security manager is present and it refuses access`

`NullPointerException - if any argument is null`

findSetter

```
public MethodHandle findSetter(Class<?> refc,
                               String name,
                               Class<?> type)
                        throws NoSuchFieldException,
                               IllegalAccessException
```

Produces a method handle giving write access to a non-static field. The type of the method handle will have a void return type. The method handle will take two arguments, the instance containing the field, and the value to be stored. The second argument will be of the field's value type. Access checking is performed immediately on behalf of the lookup class.

Parameters:

`refc - the class or interface from which the method is accessed`

`name - the field's name`

`type - the field's type`

Returns:

`a method handle which can store values into the field`

Throws:

`NoSuchFieldException - if the field does not exist`

IllegalAccessException - if access checking fails, or if the field is static

SecurityException - if a security manager is present and it refuses access

NullPointerException - if any argument is null

findStaticGetter

```
public MethodHandle findStaticGetter(Class<?> refc,
                                     String name,
                                     Class<?> type)
                              throws NoSuchFieldException,
                                     IllegalAccessException
```

Produces a method handle giving read access to a static field. The type of the method handle will have a return type of the field's value type. The method handle will take no arguments. Access checking is performed immediately on behalf of the lookup class.

If the returned method handle is invoked, the field's class will be initialized, if it has not already been initialized.

Parameters:

refc - the class or interface from which the method is accessed

name - the field's name

type - the field's type

Returns:

a method handle which can load values from the field

Throws:

NoSuchFieldException - if the field does not exist

IllegalAccessException - if access checking fails, or if the field is not static

SecurityException - if a security manager is present and it refuses access

NullPointerException - if any argument is null

findStaticSetter

```
public MethodHandle findStaticSetter(Class<?> refc,
                                     String name,
                                     Class<?> type)
```

```
                                throws NoSuchFieldException,
                                       IllegalAccessException
```

Produces a method handle giving write access to a static field. The type of the method handle will have a void return type. The method handle will take a single argument, of the field's value type, the value to be stored. Access checking is performed immediately on behalf of the lookup class.

If the returned method handle is invoked, the field's class will be initialized, if it has not already been initialized.

Parameters:

`refc - the class or interface from which the method is accessed`

`name - the field's name`

`type - the field's type`

Returns:

`a method handle which can store values into the field`

Throws:

`NoSuchFieldException - if the field does not exist`

`IllegalAccessException - if access checking fails, or if the field is not static`

`SecurityException - if a security manager is present and it refuses access`

`NullPointerException - if any argument is null`

bind

```
public MethodHandle bind(Object receiver,
                         String name,
                         MethodType type)
                  throws NoSuchMethodException,
                         IllegalAccessException
```

Produces an early-bound method handle for a non-static method. The receiver must have a supertype `defc` in which a method of the given name and type is accessible to the lookup class. The method and all its argument types must be accessible to the lookup object. The type of the method handle will be that of the method, without any insertion of an additional receiver parameter. The given receiver will be bound into the method handle, so that every call to the method handle will invoke the requested method on the given receiver.

The returned method handle will have variable arity if and only if the method's variable arity modifier bit (`0x0080`) is set *and* the trailing array

argument is not the only argument. (If the trailing array argument is the only argument, the given receiver value will be bound to it.)

This is equivalent to the following code:

```
import static java.lang.invoke.MethodHandles.*;
import static java.lang.invoke.MethodType.*;
...
MethodHandle mh0 = lookup().findVirtual(defc, name, type);
MethodHandle mh1 = mh0.bindTo(receiver);
MethodType mt1 = mh1.type();
if (mh0.isVarargsCollector())
  mh1 = mh1.asVarargsCollector(mt1.parameterType(mt1.parameterCount()-1));
return mh1;
```

where `defc` is either `receiver.getClass()` or a super type of that class, in which the requested method is accessible to the lookup class. (Note that `bindTo` does not preserve variable arity.)

Parameters:

`receiver` - the object from which the method is accessed

`name` - the name of the method

`type` - the type of the method, with the receiver argument omitted

Returns:

the desired method handle

Throws:

`NoSuchMethodException` - if the method does not exist

`IllegalAccessException` - if access checking fails or if the method's variable arity modifier bit is set and asVarargsCollector fails

`SecurityException` - if a security manager is present and it refuses access

`NullPointerException` - if any argument is null

See Also:

`MethodHandle.bindTo(java.lang.Object)`, `findVirtual(java.lang.Class<?>, java.lang.String, java.lang.invoke.MethodType)`

unreflect

```
public MethodHandle unreflect(Method m)
                       throws IllegalAccessException
```

Makes a direct method handle to *m*, if the lookup class has permission. If *m* is non-static, the receiver argument is treated as an initial argument. If *m* is virtual, overriding is respected on every call. Unlike the Core Reflection API, exceptions are *not* wrapped. The type of the method handle will be that of the method, with the receiver type prepended (but only if it is non-static). If the method's `accessible` flag is not set, access checking is performed immediately on behalf of the lookup class. If *m* is not public, do not share the resulting handle with untrusted parties.

The returned method handle will have variable arity if and only if the method's variable arity modifier bit (`0x0080`) is set.

If *m* is static, and if the returned method handle is invoked, the method's class will be initialized, if it has not already been initialized.

Parameters:

`m - the reflected method`

Returns:

`a method handle which can invoke the reflected method`

Throws:

`IllegalAccessException - if access checking fails or if the method's variable arity modifier bit is set and asVarargsCollector fails`

`NullPointerException - if the argument is null`

unreflectSpecial

```
public MethodHandle unreflectSpecial(Method m,
                                     Class<?> specialCaller)
                              throws IllegalAccessException
```

Produces a method handle for a reflected method. It will bypass checks for overriding methods on the receiver, as if called from an `invokespecial` instruction from within the explicitly specified `specialCaller`. The type of the method handle will be that of the method, with a suitably restricted receiver type prepended. (The receiver type will be `specialCaller` or a subtype.) If the method's `accessible` flag is not set, access checking is performed immediately on behalf of the lookup class, as if `invokespecial` instruction were being linked.

Before method resolution, if the explicitly specified caller class is not identical with the lookup class, or if this lookup object does not have private access privileges, the access fails.

The returned method handle will have variable arity if and only if the method's variable arity modifier bit (`0x0080`) is set.

Parameters:

m - the reflected method

specialCaller - the class nominally calling the method

Returns:

a method handle which can invoke the reflected method

Throws:

IllegalAccessException - if access checking fails or if the method's variable arity modifier bit is set and asVarargsCollector fails

NullPointerException - if any argument is null

unreflectConstructor

```
public MethodHandle unreflectConstructor(Constructor<?> c)
                                  throws IllegalAccessException
```

Produces a method handle for a reflected constructor. The type of the method handle will be that of the constructor, with the return type changed to the declaring class. The method handle will perform a `newInstance` operation, creating a new instance of the constructor's class on the arguments passed to the method handle.

If the constructor's `accessible` flag is not set, access checking is performed immediately on behalf of the lookup class.

The returned method handle will have variable arity if and only if the constructor's variable arity modifier bit (`0x0080`) is set.

If the returned method handle is invoked, the constructor's class will be initialized, if it has not already been initialized.

Parameters:

c - the reflected constructor

Returns:

a method handle which can invoke the reflected constructor

Throws:

IllegalAccessException - if access checking fails or if the method's variable arity modifier bit is set and asVarargsCollector fails

NullPointerException - if the argument is null

unreflectGetter

```
public MethodHandle unreflectGetter(Field f)
                             throws IllegalAccessException
```

Produces a method handle giving read access to a reflected field. The type of the method handle will have a return type of the field's value type. If the field is static, the method handle will take no arguments. Otherwise, its single argument will be the instance containing the field. If the field's `accessible` flag is not set, access checking is performed immediately on behalf of the lookup class.

If the field is static, and if the returned method handle is invoked, the field's class will be initialized, if it has not already been initialized.

Parameters:

`f - the reflected field`

Returns:

`a method handle which can load values from the reflected field`

Throws:

`IllegalAccessException - if access checking fails`

`NullPointerException - if the argument is null`

unreflectSetter

```
public MethodHandle unreflectSetter(Field f)
                             throws IllegalAccessException
```

Produces a method handle giving write access to a reflected field. The type of the method handle will have a void return type. If the field is static, the method handle will take a single argument, of the field's value type, the value to be stored. Otherwise, the two arguments will be the instance containing the field, and the value to be stored. If the field's `accessible` flag is not set, access checking is performed immediately on behalf of the lookup class.

If the field is static, and if the returned method handle is invoked, the field's class will be initialized, if it has not already been initialized.

Parameters:

`f - the reflected field`

Returns:

`a method handle which can store values into the reflected field`

Throws:

`IllegalAccessException - if access checking fails`

`NullPointerException - if the argument is null`

revealDirect

```
public MethodHandleInfo revealDirect(MethodHandle target)
```

Cracks a direct method handle created by this lookup object or a similar one. Security and access checks are performed to ensure that this lookup object is capable of reproducing the target method handle. This means that the cracking may fail if target is a direct method handle but was created by an unrelated lookup object. This can happen if the method handle is caller sensitive and was created by a lookup object for a different class.

Parameters:

`target - a direct method handle to crack into symbolic reference components`

Returns:

`a symbolic reference which can be used to reconstruct this method handle from this lookup object`

Throws:

`SecurityException - if a security manager is present and it refuses access`

`IllegalArgumentException - if the target is not a direct method handle or if access checking fails`

`NullPointerException` - if the target is null

Since:

1.8

See Also:

`MethodHandleInfo`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.